

DATA STRUCTURE

LAB PROGRAMS

14. Write a C program for binary tree construction and all tree traversing .

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node
```

```
{
```

```
    int data;
```

```
    struct Node *left;
```

```
    struct Node *right;
```

```
};
```

```
struct Node* createNode(int value)
```

```
{
```

```
    struct Node *newNode;
```

```
    newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = value;
```

```
    newNode->left = NULL;
```

```
    newNode->right = NULL;
```

```
    return newNode;
```

```
}
```

```
struct Node* insert(struct Node *root, int value)
```

```
{
    if(root == NULL)
        return createNode(value);

    if(value < root->data)
        root->left = insert(root->left, value);
    else
        root->right = insert(root->right, value);

    return root;
}
```

```
void inorder(struct Node *root)
{
    if(root != NULL)
    {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
```

```
void preorder(struct Node *root)
{
    if(root != NULL)
    {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}
```

```
    }  
}
```

```
void postorder(struct Node *root)
```

```
{  
    if(root != NULL)  
    {  
        postorder(root->left);  
        postorder(root->right);  
        printf("%d ", root->data);  
    }  
}
```

```
int main()
```

```
{  
    struct Node *root = NULL;  
    int n, value, i;  
  
    printf("Enter number of nodes: ");  
    scanf("%d", &n);  
  
    printf("Enter node values: ");  
    for(i = 0; i < n; i++)  
    {  
        scanf("%d", &value);  
        root = insert(root, value);  
    }
```

```
    printf("Inorder Traversal: ");
```

```
inorder(root);

printf("\nPreorder Traversal: ");
preorder(root);

printf("\nPostorder Traversal: ");
postorder(root);

return 0;
}
```

Output

```
Enter number of nodes: 7
Enter node values: 50 30 70 20 40 60 80
Inorder Traversal: 20 30 40 50 60 70 80
Preorder Traversal: 50 30 20 40 70 60 80
Postorder Traversal: 20 40 30 60 80 70 50

=== Code Execution Successful ===
```